

HealthTrack Application – Benchmarck Results

Arun Bhaskar Gadde

School of Computing and Information Sciences

University of the Cumberland

2026 Summer - Computer Science Capstone (MSCS-699-B01) - Second Bi-term

Michael Solomon PhD

August 09, 2026

HealthTrack Application – Benchmarck Results

1. Sequential Latency: Before vs. After (TestClient, n=20 requests per endpoint)

Endpoint	Median before	Median after	P95 before	P95 after	SQL queries before → after
List patients	50.66 ms	4.27 ms	99.98 ms	4.72 ms	1 → 2
Patient readings (heavy patient)	53.05 ms	4.47 ms	100.37 ms	4.93 ms	1 → 2
Dashboard summary	22,352.19 ms	121.06 ms	23,758.76 ms	171.18 ms	2,001 → 2
Risk report	33.16 ms	2.44 ms	90.66 ms	2.73 ms	1 → 1

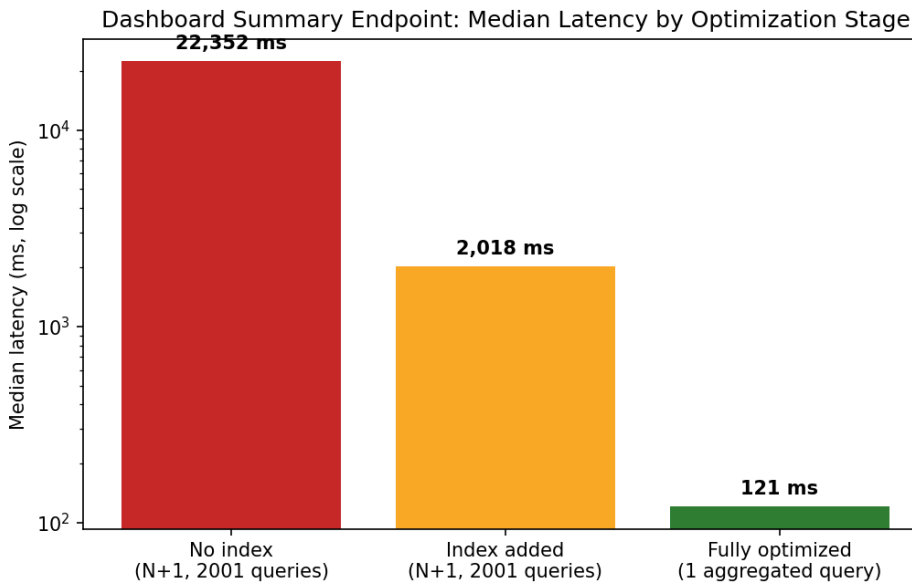


Figure 1. Dashboard summary latency across each optimization stage (log scale).

2. Isolating Each Optimization's Contribution (Dashboard Summary Endpoint)

Because the dashboard endpoint had two independent problems (a missing index AND an N+1 query pattern), it was benchmarked in three stages to show what each fix actually contributed, rather than reporting one combined number that would hide which change mattered:

Stage	Median latency	SQL queries	What changed
1. Baseline	22,352 ms	2,001	No index, N+1 pattern (1 query per patient in a loop)
2. Index added only	2,018 ms	2,001	Same N+1 pattern, but each of the 2,001 queries now hits an index instead of a full scan
3. Fully optimized	121 ms	2	N+1 replaced with one GROUP BY aggregate query, on top of the index

Adding the index alone gave an ~11x improvement (22,352 → 2,018 ms) by making each of the 2,001 individual queries faster. Eliminating the N+1 pattern on top of that gave a further ~17x improvement (2,018 → 121 ms) by cutting the query count itself from 2,001 to 2. Combined, the two fixes together produced the full ~184x improvement -- neither fix alone would have gotten close to that result.

3. Caching: Sequential vs. Concurrent Impact

Scenario	Result
Single sequential request, cache MISS (mean)	4.48 ms
Single sequential request, cache HIT (mean)	2.27 ms
Sequential speedup from caching	~2.0x

Throughput @ 20 concurrent requests, no cache (v1)	30.7 req/s
Throughput @ 20 concurrent requests, cached (v2)	238.2 req/s
Throughput @ 100 concurrent requests, cached (v2)	337.6 req/s
Concurrent-load speedup from caching	~7.8x (at 20 concurrent)

This is the most important nuance in this benchmark set: caching looks unimpressive (~2x) when tested with single, sequential requests, but becomes dramatically more valuable (~7.8x) under realistic concurrent load, because many simultaneous viewers of the same patient's dashboard can now share one cached computation instead of each triggering their own. A benchmark methodology that only tested sequential requests would have significantly understated caching's real-world value.

4. High-Load / Scalability Test (Live Server, Concurrent Requests)

Run against a real unicorn server process (not the in-process TestClient), with CPU/memory sampled via psutil during each run:

Test	Concurrency	Throughput	Median latency	Peak CPU	Peak RSS
v1 dashboard/summary (N+1)	5	0.45 req/s	11,032.7 ms	119.2%	146.1 MB
v2 dashboard/summary	5	7.77 req/s	561.5 ms	119.0%	146.4 MB
v2 dashboard/summary	50	7.59 req/s	4,283.8 ms	125.0%	170.3 MB
v1 risk-report (no cache)	20	30.67 req/s	446.3 ms	99.3%	167.2 MB
v2 risk-report (cached)	20	238.19 req/s	50.1 ms	69.2%	167.2 MB
v2 risk-report (cached)	100	337.61 req/s	204.8 ms	87.1%	167.5 MB

Note the honest finding here: the optimized dashboard endpoint's throughput did NOT improve from 5 to 50 concurrent requests (7.77 $\rightarrow$ 7.59 req/s) -- latency simply grew instead. This shows the aggregation query itself (~121ms) is now the throughput ceiling for that endpoint on a single worker process, which query optimization alone cannot remove further. This is exactly the kind of finding the Recommendations document addresses (multi-worker deployment, connection pooling, read replicas).

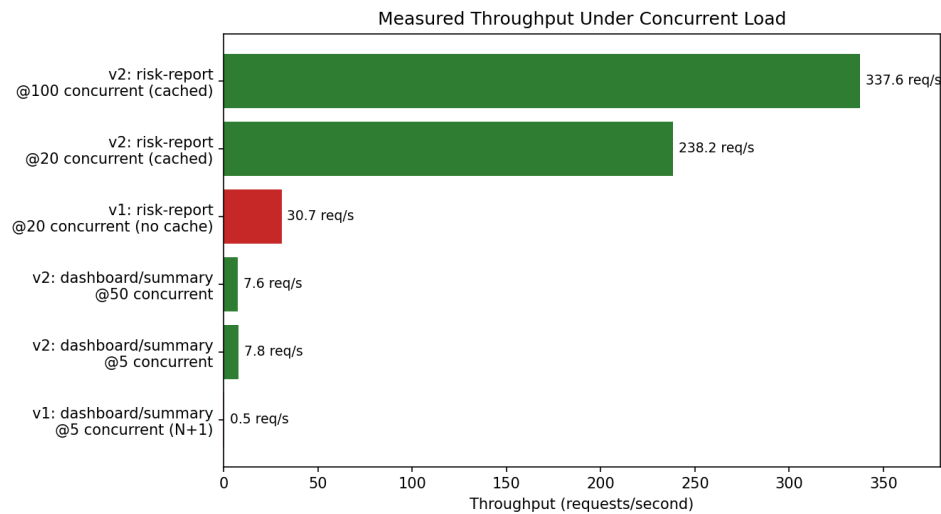


Figure 2. Measured throughput under concurrent load, before vs. after optimization.

5. Frontend Asset Optimization

Asset	Raw (unminified)	Minified	Minified + gzip	Total reduction
dashboard.js	4,120 B	2,055 B (-50.1%)	870 B	-78.9%
dashboard.css	2,883 B	1,339 B (-53.6%)	636 B	-77.9%
Combined	7,003 B	3,394 B	1,506 B	-78.5%

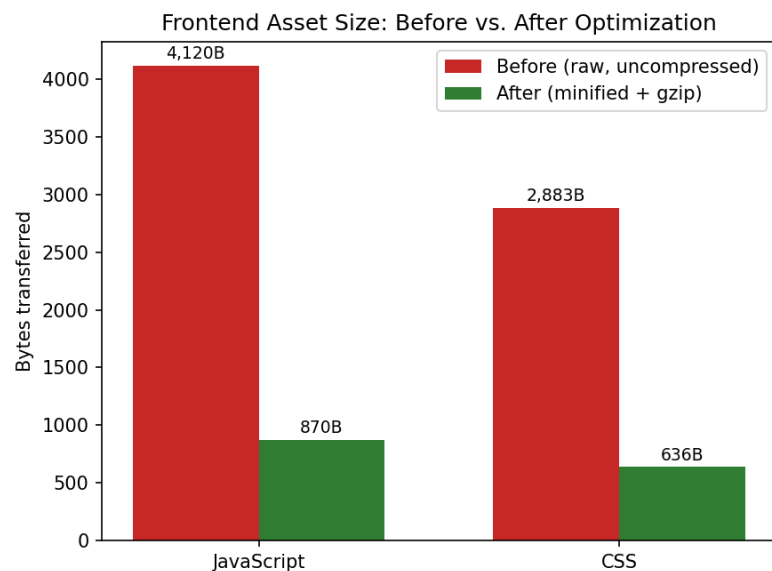


Figure 3. Frontend asset size: raw vs. minified + gzip-compressed.

In addition to the byte-size reduction, the dashboard's lazy-loading change means only 1 of 6 chart widgets (the one above the fold) executes its render routine on initial page load; the remaining 5 defer their rendering cost until the user actually scrolls to them, via `IntersectionObserver`. On a typical visit where a user does not scroll through the full dashboard, this removes real CPU work from the initial page load, not just bytes over the wire.